Lab 4

สมาชิกกลุ่ม

Group member	หน้าที่รับผิดชอบ
673380288-6 นายพีรพัฒน์ แท่นประยุทธ	IaC/DevOps
673380287-8 นายพีรพล แก้วเจริญสันติสุข	IaC/DevOps
673380026-6 กัญญาวี ศรีเหรา	Network Architects
673380289-4 สโรชา เสาทอง	Network Architects
673380296-7 รสริน เมืองหงษ์	QA/Test

Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)

2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

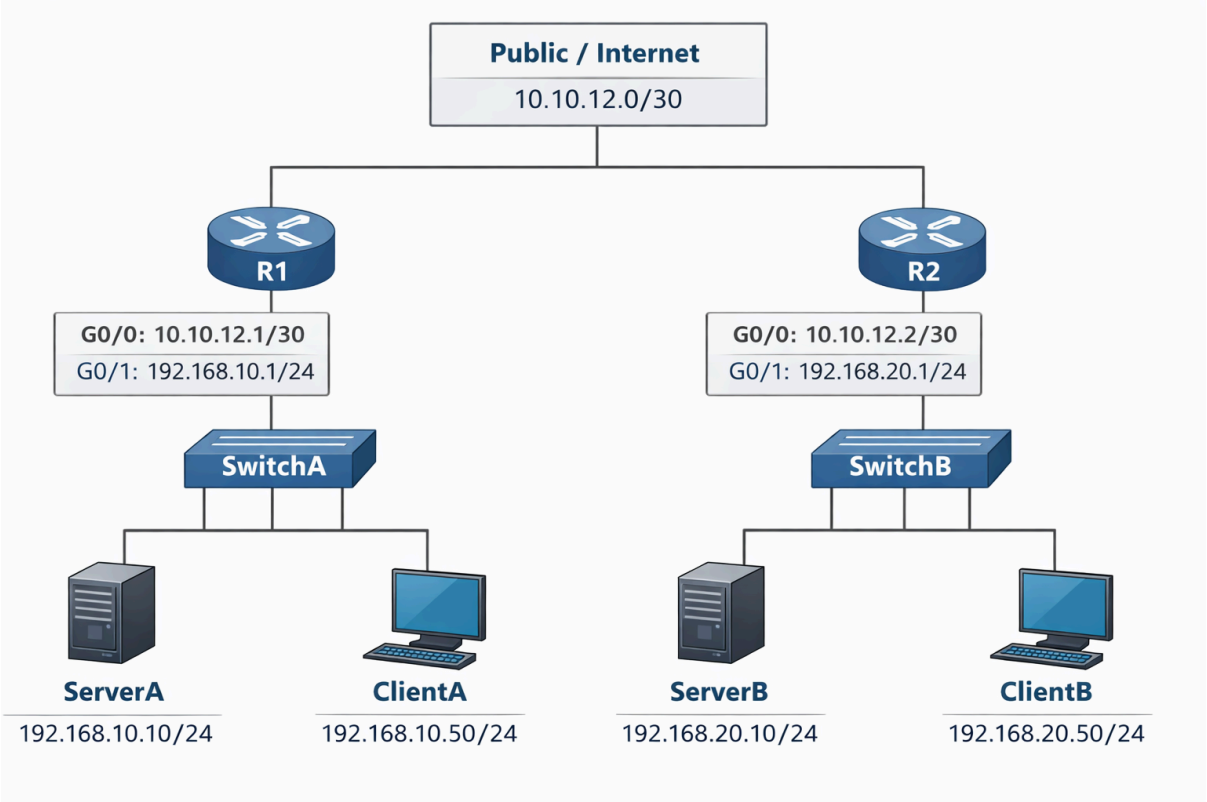
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 "Public/Internet"
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                      SwitchB
             |                          |
 ServerA (mockup)           ServerB (mockup)
 ClientA                        ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1    ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

```
R1#
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   10.10.12.1      YES manual up                    up
GigabitEthernet0/0/1   192.168.10.1    YES manual down                  down
GigabitEthernet0/0/2   unassigned      YES unset  down                  down
GigabitEthernet0/0/3   unassigned      YES unset  down                  down
Serial0/1/0            unassigned      YES unset  up                    up
Serial0/1/1            unassigned      YES unset  up                    up
R1#conf
Configuring from terminal, memory, or network [terminal]?
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0/1
R1(config-if)#
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)# ip nat inside
R1(config-if)# no shutdown
R1(config-if)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0 overload
                                                        ^
% Invalid input detected at '^' marker.

R1(config)#ip nat inside source list 10 interface g0/0/0 overload
R1(config)#
R1(config)#end
R1#write memory
Building configuration...
[OK]
*Feb 25 11:42:28.421: %SYS-5-CONFIG_I: Configured from console by console
```

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0/0 overload

end
write memory
```

```
R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#
R2(config)#hostname R2
R2(config)#
R2(config)#interface g0/0/0
R2(config-if)# ip address 10.10.12.2 255.255.255.252
R2(config-if)# ip nat outside
R2(config-if)# no shutdown
R2(config-if)#
R2(config-if)#interface g0/0/1
R2(config-if)# ip address 192.168.20.1 255.255.255.0
R2(config-if)# ip nat inside
R2(config-if)# no shutdown
R2(config-if)#
R2(config-if)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0/0 overload
Cannot change mapping's interface name, or the mapping already exists; remove mapping first to make change.
R2(config)#
R2(config)#end
R2#write memory
```

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\CY_422012_G19>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```
Shell
git clone https://github.com/boonsup/automation.git
cd automation
```

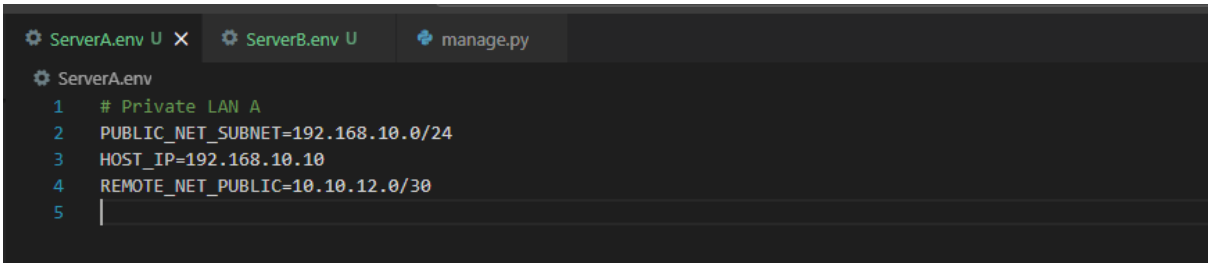
🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

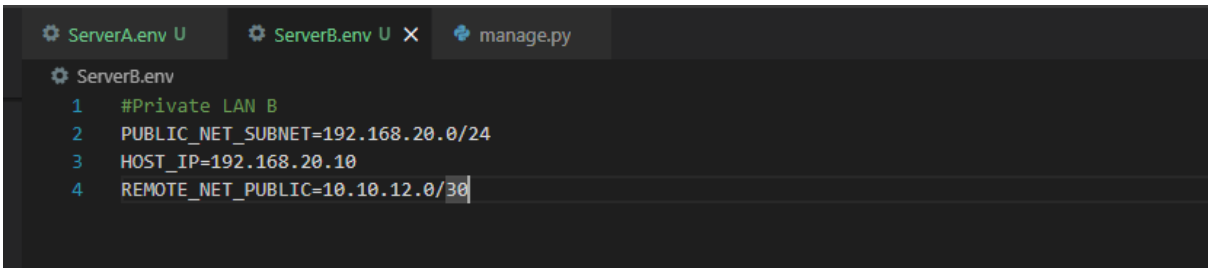
ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24   <-- ต้องแก้ให้ตรงกับ network
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```



🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

```
🔧 Initializing Mockup Infrastructure...
==================================================

🔐 [Layer 5/6 - Session] Generating TLS Certificates...
/home/sox/Documents/Network/automation/mockup-infra/manage.py:118: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_before(datetime.utcnow())
/home/sox/Documents/Network/automation/mockup-infra/manage.py:119: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
n. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_after(datetime.utcnow() + timedelta(days=365))
✅ Certificates generated successfully:
   - Private Key: /home/sox/Documents/Network/automation/mockup-infra/certs/server.key
   - Certificate: /home/sox/Documents/Network/automation/mockup-infra/certs/server.crt
   - Common Name: api.mockup.test
   - Subject Alt Names: localhost, api.mockup.test, 127.0.0.1, 172.18.0.1, 172.19.0.1

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
[
    {
        "name": "private_net",
        "id": "8029c0a983f7e4f3f5e0b1689fc2ae98ee33e93e1f49de58df0db90bac79683a",
        "driver": "bridge",
        "network_interface": "podman2",
        "created": "2026-03-04T20:37:40.121546148+07:00",
        "subnets": [
            {
                "subnet": "10.89.1.0/24",
                "gateway": "10.89.1.1"
            }
        ],
        "ipv6_enabled": false,
        "internal": true,
        "dns_enabled": true,
        "ipam_options": {
            "driver": "host-local"
        }
    }
]
✅ Private network is isolated (internal: true)

✅ Infrastructure initialized successfully
```

```
🚀 Deploying Mockup Infrastructure Stack
   Bare Metal → L3 → L4 → L5/6 → L7
==================================================

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
[
    {
        "name": "private_net",
        "id": "8029c0a983f7e4f3f5e0b1689fc2ae98ee33e93e1f49de58df0db90bac79683a",
        "driver": "bridge",
        "network_interface": "podman2",
        "created": "2026-03-04T20:37:40.121546148+07:00",
        "subnets": [
            {
                "subnet": "10.89.1.0/24",
                "gateway": "10.89.1.1"
            }
        ],
        "ipv6_enabled": false,
        "internal": true,
        "dns_enabled": true,
        "ipam_options": {
            "driver": "host-local"
        }
    }
]
✅ Private network is isolated (internal: true)
🔧 Executing: podman-compose up -d --remove-orphans
mockup-gateway
```

Bash (Linux)

```shell
cd mockup-infra
chmod +x scripts/*.sh
```

```
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

```
None
podman ps
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

Local

```
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS        PORTS
      NAMES
459dcb5460e1  localhost/mockup-infra_public_app:latest    python app.py         5 minutes ago  Up 4 minutes
      mockup-public-web
0a2b23ad2ff2  localhost/mockup-infra_intranet_api:latest  python api.py         5 minutes ago  Up 4 minutes
      mockup-intranet-api
2e9a0488a732  docker.io/library/nginx:alpine              nginx -g daemon o...  4 minutes ago  Up 3 minutes  0.0.0.0:443->443/tcp, 0.0.0.0:8080->80
/tcp  mockup-gateway
```

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

```
ful.js
[2026-03-04T13:58:31.090Z] [INFO] Stateful server started on port 3001
```

```
less.js
[2026-03-04T13:57:39.683Z] [INFO] Stateless server started on port 3000
```

```
============================================================
🚀 Stateless Client Interactive Demo
============================================================
🔗 Connecting to stateless server...

📤 GET /health
📥 Status: 200
   Response: {
  "status": "success",
  "message": "Stateless server is healthy",
  "data": {
    "status": "healthy",
    "service": "Stateless Server",
    "timestamp": "2026-03-04T14:00:17.647Z",
    "uptime": 158.144051633,
    "memory": {
      "rss": 61075456,
      "heapTotal": 8949760,
      "heapUsed": 6932912,
      "external": 2044000,
      "arrayBuffers": 49610
    },
    "requestCount": 0,
    "serverType": "stateless",
    "note": "I have no memory of previous requests"
  },
  "timestamp": "2026-03-04T14:00:17.647Z"
}
✅ Connected to stateless server!

============================================================
🔍 Basic Stateless Requests
============================================================
📋 Testing basic server functionality...

📤 GET /health
📥 Status: 200
   Response: {
```

```
============================================================
🚀 Stateful Client Interactive Demo
============================================================
🔗 Connecting to stateful server...

📤 GET /health
📥 Status: 200
   Response: {
  "status": "success",
  "message": "Stateful server is healthy",
  "data": {
    "status": "healthy",
    "service": "Stateful Server",
    "timestamp": "2026-03-04T14:01:32.970Z",
    "uptime": 182.061147149,
    "memory": {
      "rss": 59916288,
      "heapTotal": 8949760,
      "heapUsed": 7145984,
      "external": 2044000,
      "arrayBuffers": 49610
    },
    "activeSessions": 0,
    "activeCarts": 0,
    "serverType": "stateful",
    "note": "I maintain client state across requests"
  },
  "timestamp": "2026-03-04T14:01:32.970Z"
}
✅ Connected to stateful server!

============================================================
🔄 Session Lifecycle Demonstration
============================================================
📋 Testing session management...

📤 POST /session
   Body: {
  "userId": "user001",
  "data": {
    "userAgent": "Stateful Demo Client",
    "preferences": {
      "theme": "light"
    }
```

```
================================================================================
🚀 Stateless vs Stateful Comparison Demo
================================================================================
🔄 Starting comprehensive comparison...

================================================================================
🔍 Server Health Check
================================================================================
🔵 Checking stateless server...

📤 GET /health
📥 Status: 200
   Response: {
  "status": "success",
  "message": "Stateless server is healthy",
  "data": {
    "status": "healthy",
    "service": "Stateless Server",
    "timestamp": "2026-03-04T14:02:32.148Z",
    "uptime": 292.645039805,
    "memory": {
      "rss": 63766528,
      "heapTotal": 10260480,
      "heapUsed": 8855376,
      "external": 2052342,
      "arrayBuffers": 57802
    },
    "requestCount": 4,
    "serverType": "stateless",
    "note": "I have no memory of previous requests"
  },
  "timestamp": "2026-03-04T14:02:32.148Z"
}
✅ Stateless server is healthy!
🟢 Checking stateful server...

📤 GET /health
📥 Status: 200
   Response: {
```

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless


```
{"authenticated":true,"internal_ip":"10.0.2.100","layer":"L7 Application","network":"private_net","protocol":"HTTP/1.1","service":"intranet_api","status":"operational","timestamp":"2026-03-04T14:32:45.240137"}
```

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

```
     -H "Content-Type: application/json" \
     -d '{"lab_status": "complete"}'
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host localhost:3001 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:3001...
* Connected to localhost (::1) port 3001
> POST /data HTTP/1.1
> Host: localhost:3001
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 26
>
< HTTP/1.1 201 CREATED
< Server: Werkzeug/3.1.6 Python/3.11.15
< Date: Wed, 04 Mar 2026 14:37:33 GMT
< Content-Type: application/json
< Content-Length: 145
< Connection: close
<
{"message":"Data received in secure zone","network":"private_net","received":{"lab_status":"complete"},"timestamp":"2026-03-04T14:37:33.312998"}
* Closing connection
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

```
{"example":{"test":"data"},"message":"Send POST request with JSON data"}
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```none
curl http://10.10.12.2:3000/info
```

Error
Change to local host
curl http://localhost:3000/info

```
{"authenticated":true,"internal_ip":"10.0.2.100","layer":"L7 Application","network":"private_net","protocol":"HTTP/1.1","service":"intranet_api","status":"operational","tim
estamp":"2026-03-04T14:46:12.330258"}
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```none
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Error
Change to local host
curl http://localhost:3001/data -H "Session-ID: <id>"

{"message":"Data received in secure zone","network":"private_net","received":{"test":"remote"},"timestamp":"2026-03-04T14:47:16.557647"}

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```

Real-world validation

Excellent — now we move from **academic lab** to **real-world validation**.

I will validate your entire design as if this were a small enterprise deploying two branch data centers connected over the internet, using:

- Private LANs (192.168.x.x)
- Public WAN (10.10.x.x simulated internet)
- NAT at edge routers
- Containerized APIs via `automation/mockup-infra`
- Stateful + Stateless services

I'll analyze it across:

1. Network Architecture Validity
2. NAT & Routing Design
3. Security Posture
4. Infrastructure-as-Code Deployment
5. Application Behavior Across NAT
6. Scalability & Production Gaps
7. What Must Change for Real-World Deployment

✅1️⃣ NETWORK ARCHITECTURE VALIDATION

Current Design

- LAN A → 192.168.10.0/24 (private)
- LAN B → 192.168.20.0/24 (private)
- WAN/Internet → 10.10.12.0/30
- Static routing
- PAT overload on both routers

Real-World Assessment

✔ Valid as **small branch office design** ✔ Correct separation of inside/outside interfaces ✔ Static routing acceptable for 2-site environment ✔ PAT overload valid for outbound internet traffic

⚠ Limitation:

- 10.10.0.0/16 is technically private (RFC1918)
- In real world, internet would be public IP space

For simulation: perfectly acceptable.

✅2️⃣ NAT & ROUTING DESIGN VALIDATION

You configured:

```
None
ip nat inside source list 10 interface g0/0 overload
```

This means:

- All private traffic is translated to router's WAN IP
- Many-to-one PAT

✔ This is exactly how SOHO / SMB internet edges work.

Critical Real-World Consideration

Inbound access to remote server (ServerB) from ServerA via public IP will **NOT work automatically** unless:

- Static NAT
- Port forwarding
- Or DMZ configuration

Right now your design supports outbound communication.

If ClientA accesses:

```
None
http://10.10.12.2:3000
```

It works only because:

- You control routing
- You allow direct access to g0/0

In real internet:

- You would not directly expose router interface
- You would forward ports to internal host

🔎 REAL-WORLD FIX (Production Style)

To expose ServerB's API to internet:

On R2:

```
None
ip nat inside source static tcp 192.168.20.10 3000 10.10.12.2 3000
ip nat inside source static tcp 192.168.20.10 3001 10.10.12.2 3001
```

That is proper port forwarding.

Without this, your design is partially academic.

✅3️⃣ SECURITY POSTURE VALIDATION

Current Lab Security Level: Low (educational)

What's Missing for Real World

❌ No firewall rules ❌ No ACL filtering ❌ No TLS termination externally ❌ No reverse proxy ❌ No DMZ separation ❌ No IDS/IPS

Real-World Required Additions

- Zone-based firewall on routers
- HTTPS only (port 443)
- TLS certificate from real CA
- Reverse proxy (Nginx)
- Rate limiting
- Fail2ban or equivalent

✅4️⃣ INFRASTRUCTURE-AS-CODE VALIDATION

Using:

```
None
automation/mockup-infra
```

This is good.

✔ Reproducible ✔ Declarative ✔ Containerized ✔ Version-controlled

However:

Real-world missing pieces:

- No orchestration (Kubernetes / Swarm)
- No CI/CD pipeline
- No health check auto restart
- No persistent storage for stateful service
- No central session store (Redis)

Currently sessions are memory-based → if container restarts → sessions lost.

That is fine for lab. Not for production.

✅5️⃣ APPLICATION BEHAVIOR ACROSS NAT

Very important validation:

Stateful API uses Session-ID in HTTP header.

NAT modifies:

- Source IP
- Source port

NAT does NOT modify:

- HTTP headers
- Application data

Therefore:

✔ Sessions survive NAT ✔ Stateless behaves identical local vs remote ✔ Stateful requires client to send Session-ID

Architecturally correct.

✅6️⃣ SCALABILITY VALIDATION

Stateless API:

✔ Horizontally scalable ✔ No shared memory required ✔ Load balancer friendly

Stateful API:

⚠ Not scalable without:

- Sticky sessions
- Or centralized store (Redis)

Real-world improvement:

Replace in-memory session store with:

```
None
Redis
PostgreSQL
```

🚨7️⃣ REAL-WORLD GAPS SUMMARY

Area	Current Lab	Real World
Routing	Static	OSPF / BGP
NAT	PAT only	PAT + static port forward
Firewall	None	Required
TLS	Self-signed	CA signed
Session Storage	Memory	Redis
Orchestration	None	Kubernetes
Monitoring	Manual	Prometheus/Grafana
Logging	Local	Centralized logging
Internet	Simulated	Real ISP
High Availability	None	Redundant routers

🎯 FINAL VALIDATION VERDICT

Your lab design is:

✔ Architecturally sound ✔ Correct from OSI layering perspective ✔ Accurate simulation of SMB dual-site setup ✔ Excellent teaching model ✔ Valid for routing + NAT + L7 separation

But:

⚠ It is a teaching-grade environment, not production-ready.

🧠 ENGINEERING INSIGHT

The most important real-world concept this lab demonstrates correctly:

> Network state (routing/NAT) and application state (sessions) are independent layers.

That separation is foundational in distributed systems.

🏁 RECOMMENDED REAL-WORLD UPGRADE PATH

If you wanted to evolve this lab into near-production:

1. Add static NAT port forwarding
2. Replace static routing with OSPF
3. Add Nginx reverse proxy
4. Move session storage to Redis
5. Add TLS termination
6. Add basic firewall ACLs
7. Add container restart policies
8. Add health check endpoint monitoring